

CodeCraze

auth.py — Endpoint Notes

register · login · refresh · logout · me · Axios Interceptor

Is PDF me kya hai:

- ✓ Endpoint 1 — /register — line by line explanation + interview Q&A;
- ✓ Pydantic Validation — frontend aur backend dono guards
- ✓ Endpoint 2 — /login — TokenResponse, bcrypt flow, comparison table
- ✓ Dependency Injection — Depends(get_db) — deep dive easy way
- ✓ Endpoint 3 — /refresh — Axios Interceptor, 401 silent handling

Endpoint 1 — /register

Code

```
@router.post(
    "/register",
    response_model=UserResponse,
    status_code=status.HTTP_201_CREATED,
    summary="Register a new account",
)
async def register(
    data: RegisterRequest,
    db: AsyncSession = Depends(get_db),
):
    user = await auth_service.register_user(db, data)
    return user
```

Line by Line — Notes

@router.post('/register') kya hai?

Jab bhi koi /register URL pe POST request bheje, ye function chalega. @router.post ek **decorator** hai jo FastAPI ko batata hai — 'is URL pe POST aaye toh yahi function call karna.'

response_model=UserResponse kya hai?

Server jo response bhejega uski **shape** define karta hai. Agar DB me password_hash pada hai toh bhi response me nahi aayega — sirf wahi fields jaayengi jo UserResponse schema me defined hain. Security + consistency ke liye.

status_code=201 — 200 kyun nahi?

HTTP convention:

200 OK → kuch fetch kiya / kaam hua

201 Created → **naya resource create hua**

Register pe naya user bana → isliye 201.

data: RegisterRequest kya kar raha hai?

FastAPI automatically request body ko parse karta hai aur RegisterRequest Pydantic schema se validate karta hai. Agar koi field missing hai ya wrong type hai → **automatically 422 error** return hoga, tu manually check nahi karega.

db: AsyncSession = Depends(get_db) kya hai?

Ye **Dependency Injection** hai. Depends(get_db) matlab FastAPI khud get_db() function chalayega aur jo DB session mile usse db variable me inject karega. Tu manually connection nahi kharega — FastAPI karega.

AsyncSession isliye kyunki DB call time lagata hai — async hone se server tab tak doosri requests handle kar sakta hai.

await auth_service.register_user(db, data) kya hai?

Actual kaam yahan hota hai — auth_service me:

1. Password bcrypt se hash hoga
2. DB me user insert hoga
3. User object return hoga

await isliye kyunki DB operation async hai — result aane tak server block nahi hoga.

Pydantic Validation — 2 Guards

Frontend = pehla guard (user experience ke liye)

Galat email daali → browser/React ne pakad liya → 'Enter a valid email' dikhta hai → request backend tak gayi hi nahi.

```
// React me
<input type="email" /> // browser khud check karta hai

// ya manually
if (!email.includes("@")) {
  setError("Enter a valid email")
}
```

Pydantic = doosra guard (security ke liye)

Koi Postman se seedha bheje toh bhi EmailStr validate karega → 422 error.

```
from pydantic import BaseModel, EmailStr

class RegisterRequest(BaseModel):
    username: str
    email: EmailStr # ← valid email format check
    password: str
```

Guard	Kahan?	Kya hota hai?
Frontend	Browser / React	Enter a valid email — request hi nahi jaati
Pydantic	FastAPI backend	EmailStr fail → 422 Unprocessable Entity

Interview Q&A; — /register

Q: /register endpoint kaise kaam karta hai?

Jab user registration form submit karta hai, frontend POST request bhejta hai /api/v1/auth/register pe — name, email, password ke saath. FastAPI RegisterRequest schema se automatically validate karta hai. Validate hone ke baad auth_service.register_user() call hota hai jahan password bcrypt se hash hota hai aur user PostgreSQL me store hota hai. Response me UserResponse schema use hota hai — password hash kabhi client ko nahi bhejte. Status code 201 return hota hai kyunki naya resource create hua.

Q: Depends(get_db) kyu use kiya directly connection lene ki jagah?

Dependency Injection pattern use kiya — isse har request pe fresh DB session milta hai, request khatam hone pe automatically close ho jaata hai, aur testing me easily mock kar sakte hain.

Q: 200 ki jagah 201 kyu?

HTTP convention — 200 OK hota hai existing operations ke liye, 201 Created specifically tab use hota hai jab server pe naya resource create ho. REST best practices follow karne ke liye.

Depends(get_db) — Deep Dive

Problem — Bina Depends ke

```
# Bina Depends ke — manually karna padta
async def register(data: RegisterRequest):
    db = get_db()           # khud connection khola
    user = await db.query(...) # kaam kiya
    db.close()              # khud close kiya
    return user             # db.close() bhool gaye? → Memory leak!
```

Har function me yahi 3 lines likhni padti — **repetition + error prone**. Agar db.close() bhool gaye → Memory leak!

Solution — Depends ke saath

```
# Depends ke saath — FastAPI sab karta hai
async def register(
    data: RegisterRequest,
    db: AsyncSession = Depends(get_db),  # ← bas itna likho
):
    user = await db.query(...)
    return user                          # close? FastAPI khud karega ■
```

Real Life Analogy

Bina Depends: Chef khud jaata hai → refrigerator se ingredients laata hai → khaana banata hai → refrigerator band karta hai.

Depends ke saath: Ek helper (FastAPI) hai — chef ke aane se pehle ingredients ready rakhta hai, kaam khatam hone pe khud refrigerator band karta hai. Chef sirf **khaana banane pe focus** karta hai.

get_db() andar kya hota hai?

```
async def get_db():
    db = SessionLocal()  # DB connection khola
    try:
        yield db          # function ko de do (yield = "le ja, wapas aane pe close karunga")
    finally:
        await db.close()  # error aaye ya na aaye — close ZAROOR hoga
```

AsyncSession — Normal vs Async

Normal Session	AsyncSession
register() chala	register() chala
DB call kiya	DB call kiya

Normal Session	AsyncSession
RUKA... wait... (blocked)	await → side me rakh
Response aaya	Doosri request handle ki ■
Aage badha	DB response aaya → wapas aaya ■

Poora Flow

```

POST /register aaya
↓
FastAPI ne dekha – "db chahiye"
↓
get_db() khud chalaya → DB connection banaya
↓
db → register() function me inject kiya
↓
register() ne kaam kiya (user banaya)
↓
Response return hua
↓
FastAPI ne automatically db.close() kiya ■

```

Interview me ek line me:

"Depends(get_db) FastAPI ka Dependency Injection hai — har request pe automatically fresh DB session inject karta hai aur request khatam hone pe khud close karta hai. AsyncSession isliye use kiya taaki DB wait ke time pe server doosri requests handle kar sake."

Endpoint 2 — /login

Code

```
@router.post(
    "/login",
    response_model=TokenResponse,
    summary="Authenticate and receive tokens",
)
async def login(
    data: LoginRequest,
    db: AsyncSession = Depends(get_db),
):
    return await auth_service.authenticate_user(db, data)
```

Line by Line — Notes

response_model=TokenResponse kya hai?

Register me UserResponse tha — user ka profile return hua.

Login me TokenResponse hai — matlab server **tokens return karega**, user object nahi.

TokenResponse me hota hai:

- access_token: str
- refresh_token: str
- token_type: str # 'bearer'

status_code kyun nahi likha yahan?

Default **200 OK** hoga — explicitly likhne ki zaroorat nahi.

Register pe 201 tha kyunki naya resource bana tha.

Login pe kuch naya create nahi hua — sirf verify karke token diya → 200 OK sahi hai.

data: LoginRequest kya hai?

Frontend se email + password aayega — Pydantic automatically validate karega.

Galat format → **422 error** automatically, backend logic tak jaayega hi nahi.

auth_service.authenticate_user() — Flow

1. Email se DB me user dhundho
↓
2. User mila? Nahi → "Invalid credentials" error
↓
3. Mila → bcrypt se password verify karo
↓
4. Password match nahi → "Invalid credentials" error
(specifically nahi bataate – security reason se)

- ↓
5. Match hua → JWT access token + refresh token banao
- ↓
6. TokenResponse return karo

Real Life Flow — CodeClash

User ne Login form fill kiya (email: test@gmail.com, password: 1234)

↓

Frontend validate kiya — email valid hai ■

↓

POST /api/v1/auth/login bheja

↓

Pydantic ne validate kiya ■

↓

auth_service.authenticate_user() chala

↓

DB me test@gmail.com dhundha → Mila → bcrypt se password verify kiya

↓

Sahi hua → JWT tokens banaye

↓

Frontend ko access_token + refresh_token mila

↓

Frontend ne token localStorage me save kiya

↓

Ab har request me Bearer token bhejega

Register vs Login — Comparison

Cheez	Register	Login
Kya karta hai	Naya user banata hai	Existing user verify karta hai
Response	UserResponse (profile)	TokenResponse (tokens)
Status Code	201 Created	200 OK
DB Operation	INSERT	SELECT
Password	Hash karke save karta	Hash se compare karta

Interview Q&A; — /login

Q: Login endpoint kaise kaam karta hai?

Frontend se email aur password POST hota hai /login pe. Pydantic LoginRequest schema se validate karta hai. Phir auth_service.authenticate_user() chalta hai — pehle DB me email se user dhundha jaata hai, phir bcrypt se password verify hota hai. Dono sahi hone pe JWT access token aur refresh token generate hote hain aur TokenResponse me return hote hain. Frontend in tokens ko store karta hai aur aage ki har protected request me Authorization: Bearer token header me

bhejta hai.

Q: Login aur Register response me kya fark hai aur kyu?

Register pe UserResponse return kiya — naya user bana toh uski profile return kari, status 201. Login pe TokenResponse return kiya — kuch naya create nahi hua, sirf authentication hua aur tokens mile, status 200. Dono alag kaam karte hain isliye response bhi alag hai.

Q: Agar galat password daala toh kya hoga?

Auth service DB me user dhundhega, milega, lekin bcrypt comparison fail hoga. Tab 401 Unauthorized return hoga — 'Invalid credentials'. Hum specifically nahi bataate ki email galat tha ya password — security reason se, attacker ko pata nahi chalna chahiye ki email exist karta hai ya nahi.

Endpoint 3 — /refresh + Axios Interceptor

User ko 401 ka pata chalega?

Nahi chalega — agar frontend sahi se banaya ho! Ye sab milliseconds me hota hai, user screen pe kuch nahi dekh raha.

```
/me call kiya
↓
401 aaya (token expire)    ← ye sab background me
↓
/refresh call kiya         ← user screen pe kuch nahi dekh raha
↓
Naya token mila
↓
/me dobara call kiya
↓
Profile data aaya ■       ← user ko lag raha hai sab normal hai
```

Axios Interceptor — Frontend me kaise handle karte hain

CodeClash me Axios use ho raha hai. Interceptor ek background watcher hota hai — har response check karta hai silently.

```
// Ye code background me silently kaam karta hai
axios.interceptors.response.use(
  (response) => response,    // sab theek → seedha return

  async (error) => {
    if (error.response.status === 401) {
      // 401 aaya → user ko mat dikhao
      // silently refresh karo
      const newToken = await refreshToken()

      // purani request naye token se dobara bhejo
      error.config.headers['Authorization'] = `Bearer ${newToken}`
      return axios(error.config)  // ← user ko pata nahi chala ■
    }
  }
)
```

Kab pata chalega user ko?

Sirf tab — jab **refresh token bhi expire** ho jaye:

```
Access token expire hua
↓
/refresh call kiya
↓
Refresh token bhi expire! (7 din baad)
↓
Ab kuch nahi kar sakte
```

↓

User ko Login page pe redirect karo

↓

"Session expire ho gayi, please login karein" ← AB DIKHTA HAI ■

Situation	Kya hota hai	User ko pata?
Access token expire	Axios silently /refresh call karta hai	Nahi ■
Naya token mila	Original request retry hoti hai	Nahi ■
Refresh token bhi expire	Login page pe redirect	Haan ■■
Refresh token chori hua	Rotation se purana invalid ho jaata hai	Nahi ■

Interview Q&A; — /refresh + Axios

Q: User ko 401 ka pata kyun nahi chalta?

Axios interceptor silently handle karta hai. Jab bhi 401 aata hai, background me /refresh call hota hai — naya token milta hai aur original request dobara bhejte hain. Itna fast hota hai ki user ko loading bhi nahi dikhti. Sirf refresh token bhi expire ho toh Login page dikhta hai.

Q: Refresh token kyu use kiya? Ek hi token kyu nahi?

Security reason se. Access token short lived hota hai — 15-30 min. Agar chura bhi liya toh thodi der me expire. Lekin user ko baar baar login nahi karna padta — refresh token long lived hota hai aur silently naya access token le leta hai. Dono milke security aur good user experience dete hain.

Q: Refresh token bhi chori ho sakta hai — toh secure kaise hai?

Refresh token HttpOnly Cookie me store karte hain — JavaScript access nahi kar sakti. Refresh Token Rotation use karte hain — har refresh pe purana token invalid ho jaata hai. Token Family technique — koi purana token dobara use kare toh SAARE tokens revoke. Dono milke security guarantee karte hain.